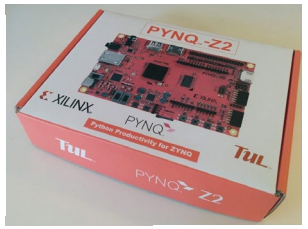
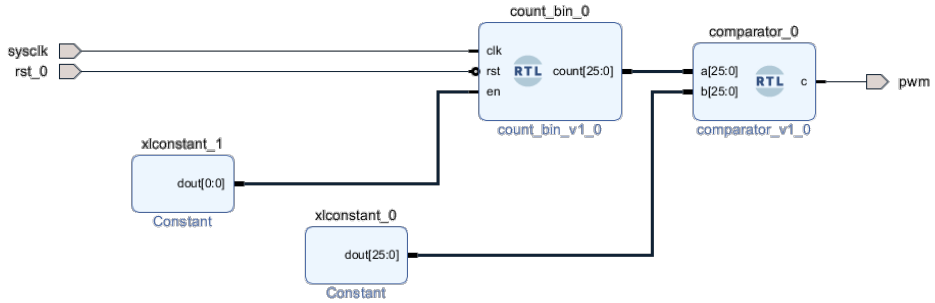


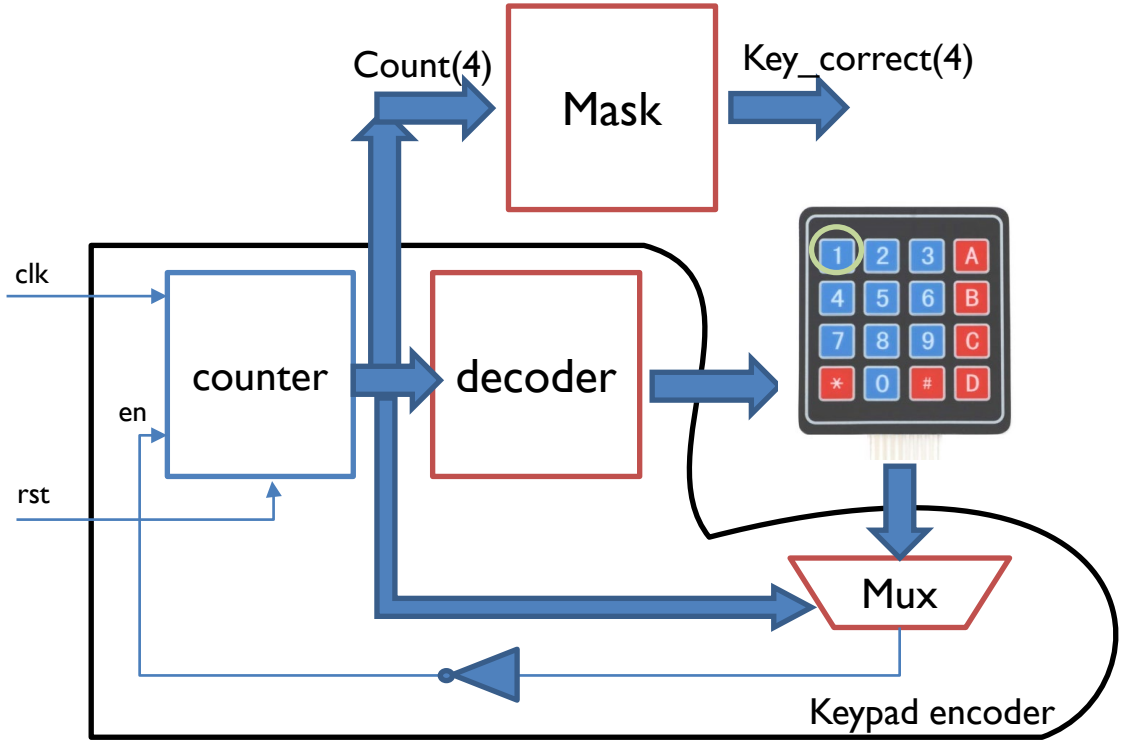
Lecture 5:
Arithmetic Units

Emad Samuel Malki Ebeid

Summary of lecture 4

Operators	VHDL
Equality & Relational	=
	/=
	<
	<=
	>
	>=
Logical	not
	and
	or





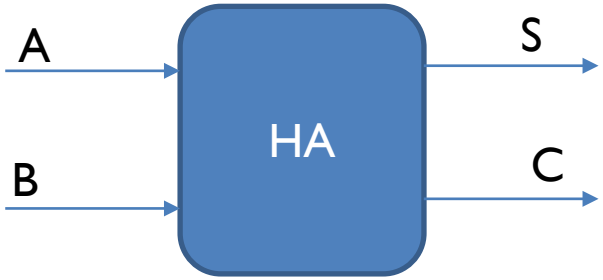
Simulation vs Synthesis

- Simulation-only operations for error messages or reading files that can be simulated, but not synthesized into circuits
- Constructs such as for-loops, while-loops, etc. are either un-synthesizable or (worse) synthesize poorly
- You need procedural code for testbench and only for testbench
- Examples:
  - AND gate is synthesizable but AND gate with 5ns wait is not synthesizable
  - Printf(”hello world”) is not synthesizable
- Use Synthesizable code to describe the function of units that will be built into hardware
- Use Non-synthesizable code to create a testbench that checks to see if your synthesizable code does what you want

Adders

- Adders are one of the basic building block for Computer-Arithmetic Logic units (ALUs) and Digital Signal Processing
- Adders operate on binary vectors.

- **Half adder (HA)**: it adds two single binary digits *A* and *B*.
  - It has two outputs, sum (*S*) and carry (*C*).
    - S = A *XOR* B
    - C = A *AND* B

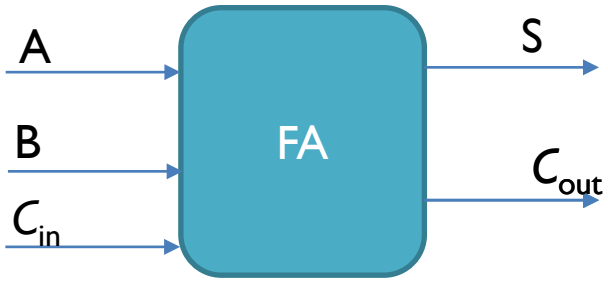


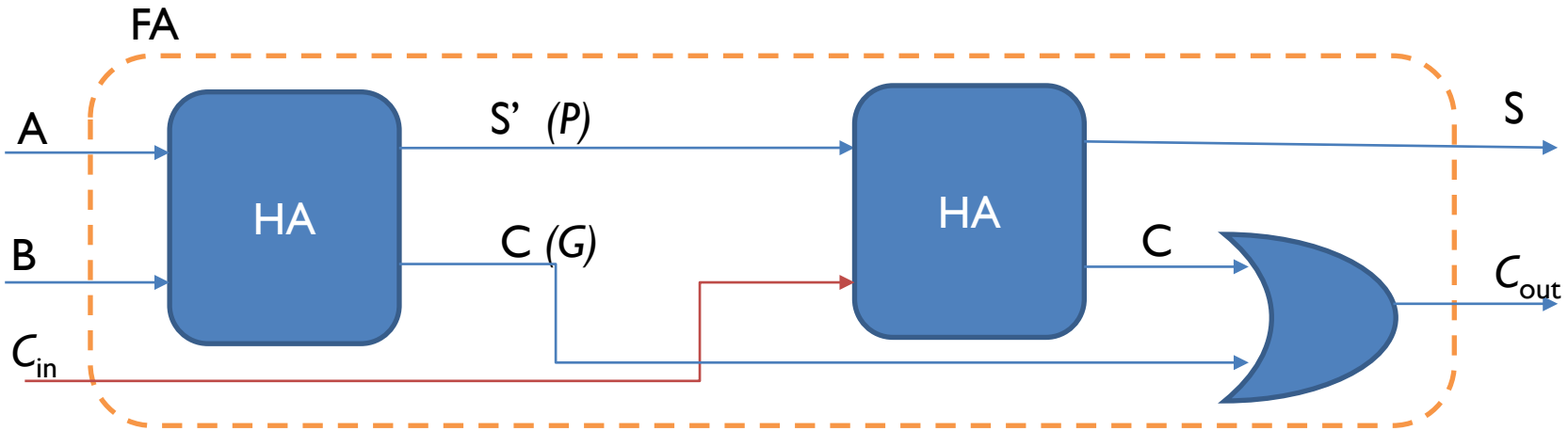
Inputs		Outputs	
A	B	C	S
0	0	0	0
1	0	0	1
0	1	0	1
1	1	1	0

Adders

Inputs			Outputs	
A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

- **Full Adder (FA):** it adds three bits, often written as *A*, *B*, and C_{in}; *A* and *B* are the operands, and C_{in} is a bit carried in from the previous less-significant stage (e.g., coming from a previous half adder).



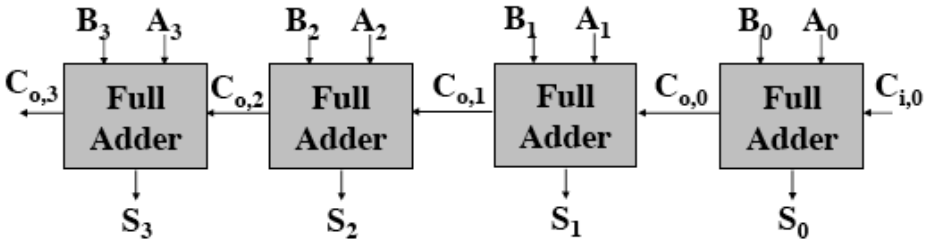


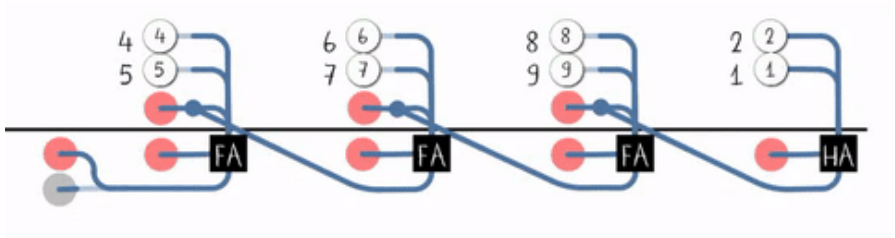
- $S = A \text{ XOR } B \text{ XOR } C_{in}$
- $C_{out} = (A \text{ AND } B) \text{ OR } (C_{in} \text{ AND } (A \text{ XOR } B))$

Adders

- **Ripple-carry adder**: an iterative array to perform binary addition, full adders chained together
  - The longest path delay is from (A_0 or B_0) to S_3,

    Or from $C_{i.0}$ to $C_{0.3}$
  - It has the advantage of requiring less hardware

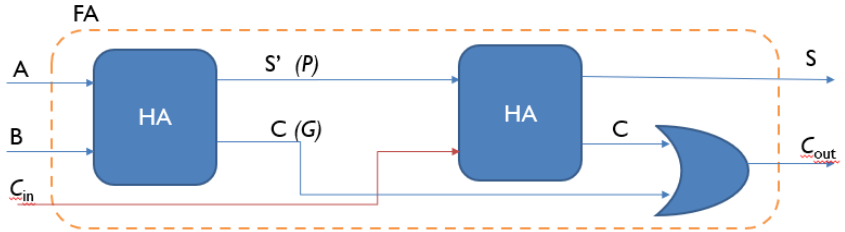




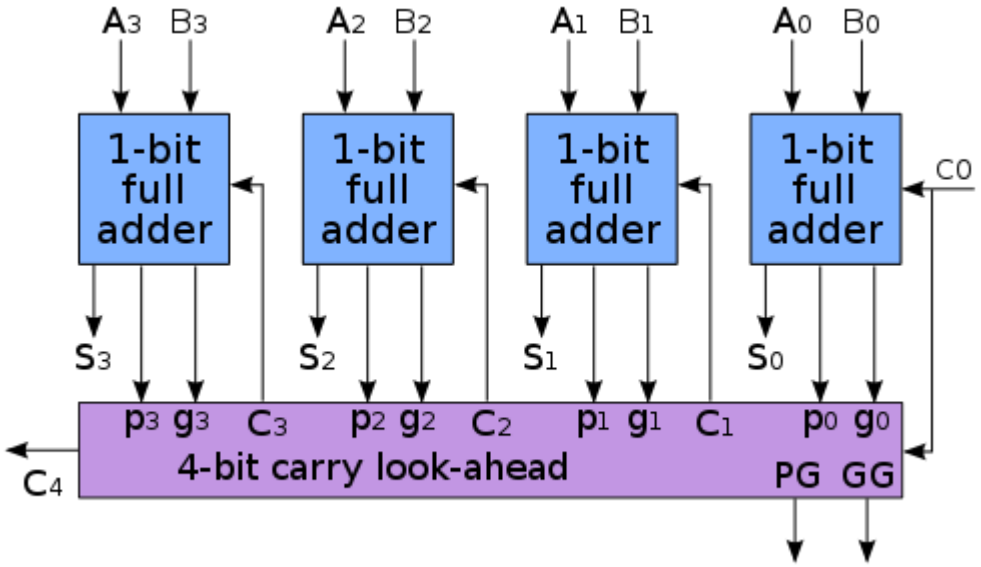
Adders

Inputs			Outpu
A	B	C_{in}	C_{out}
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

- **Carry-look-ahead adder**: a hierarchical adder to improve performance (faster than Ripple adder but takes more resources)
  - It is based on *Propagate* and *Generate* concept
    - P = A *XOR* B , G = A *AND* B
    - P and G can be calculated in advance since none of them depends on the carry bit (Cin).



$$C_1=G_0+P_0C_0$$
$$C_2=G_1+P_1C_1=G_1+P_1(G_0+P_0C_0)$$
$$=G_1+P_1G_0+P_1P_0C_0$$
$$C_3=G_2+P_2C_2=G_2+P_2(G_1+P_1G_0+P_1P_0C_0)$$
$$=G_2+P_2G_1+P_2P_1G_0+P_2P_1P_0C_0$$
$$C4=G_3+P_3C_3=G_3+P_3(G_2+P_2G_1+P_2P_1G_0+P_2P_1P_0C_0)$$
$$=G_3+P_3G_2+P_3P_2G_1+P_3P_2P_1G_0+P_3P_2P_1P_0C_0$$

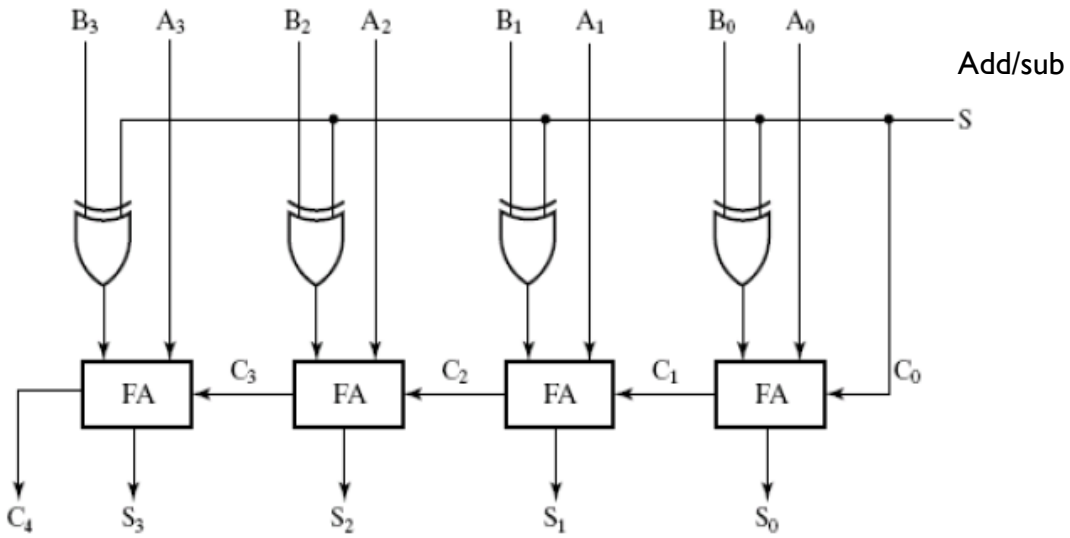


Subtractors

Subtraction by addition:

- Subtraction can be done by adding 2's complement

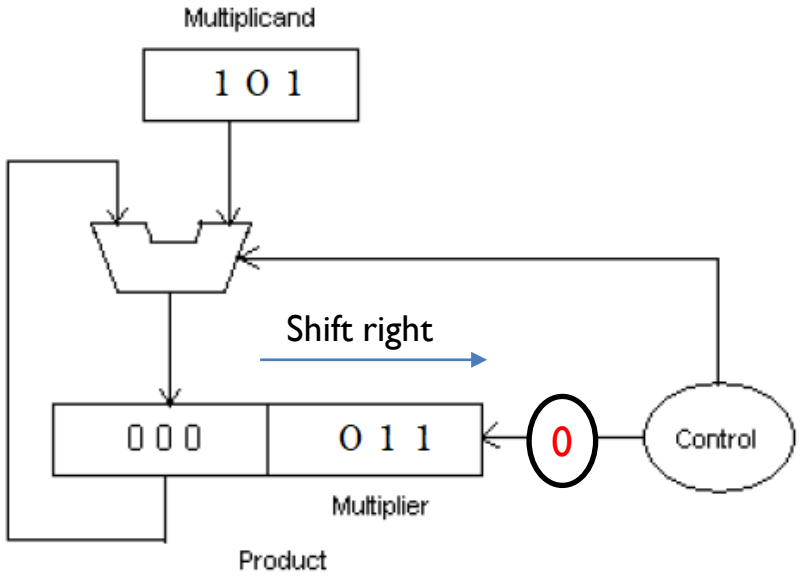
- Note:
  - A≥B => A-B
  - A<B => A-B => 2's complement



- For S=1, subtract, the 2's complement of B is formed by using XORs to form the 1's complement and adding the 1 applied to C0
- For S=0, B is passed through unchanged

Multiplication

- It can be implemented by a shift register as wide as the product and an accumulator for the partial and final product.

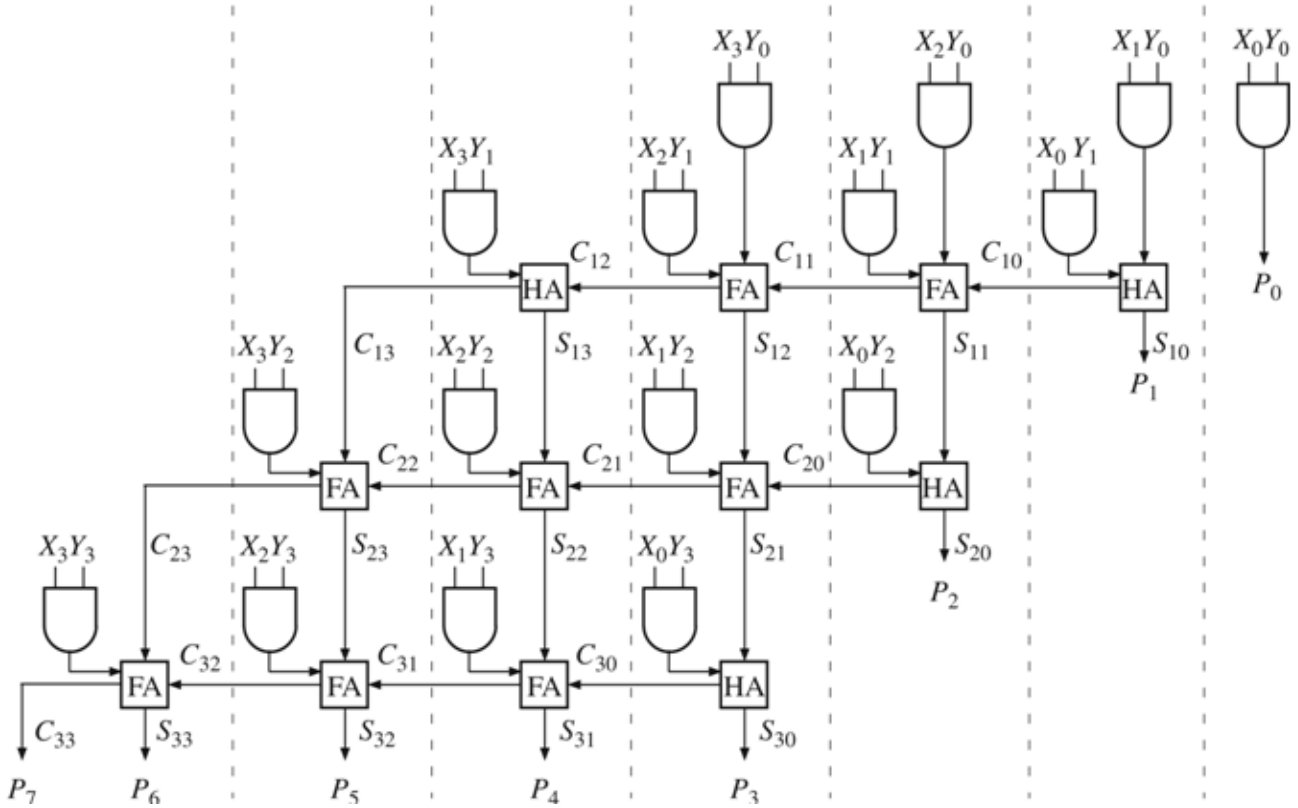


10 minutes,
Multiply: 5*3 using the shift and sum concept
(the circuit on the left)
Multiplicant (5)(101)
Multiplier (3) (011)

Array Multiplier

- It is a digital combinational circuit used for multiplying two binary numbers by employing an array of full adders and half adders.

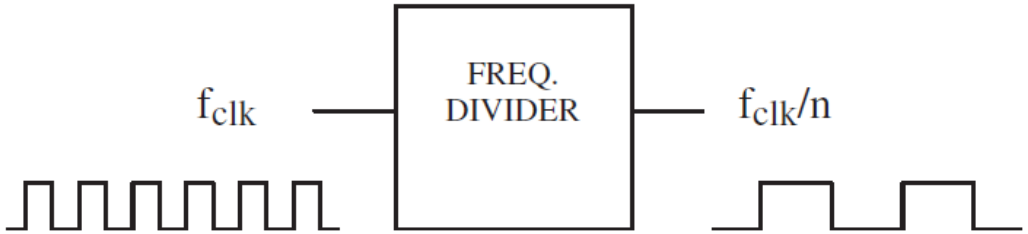
			X_3	X_2	X_1	X_0	
			Y_3	Y_2	Y_1	Y_0	
			X_3Y_0	X_2Y_0	X_1Y_0	X_0Y_0	
		X_3Y_1	X_2Y_1	X_1Y_1	X_0Y_1		
		C_{12}	C_{11}	C_{10}			
	C_{13}	S_{13}	S_{12}	S_{11}	S_{10}		
	X_3Y_2	X_2Y_2	X_1Y_2	X_0Y_2			
	C_{22}	C_{21}	C_{20}				
C_{23}	S_{23}	S_{22}	S_{21}	S_{20}			
X_3Y_3	X_2Y_3	X_1Y_3	X_0Y_3				
C_{32}	C_{31}	C_{30}					
C_{33}	S_{33}	S_{32}	S_{31}	S_{30}			
P_7	P_6	P_5	P_4	P_3	P_2	P_1	P_0



Dividers

- Note: Division is the most complex of the four basic arithmetic operations.
- Hardware solutions are correspondingly larger and more complex than the solutions for other operations, it is *best to minimize the number of divisions in any algorithm*.

- Clk divider by 2^n (PWM)



- Division by subtraction

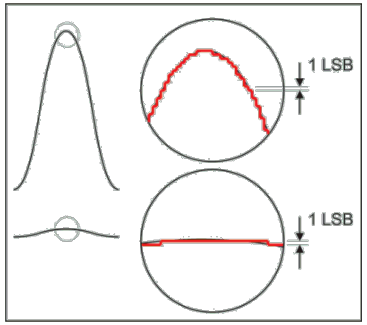
- Division by multiplication: $A^* 1/B = A^* (2^n/B)\ 2^n => \frac{10}{4} = 10\frac{2^2/4}{2^2} = 10\frac{1}{2^2} = 10.10$

Binary $a_5a_4a_3a_2a_1a_0$
000000
000001
000010
000011
000100
000101
000110
000111
001000
001001
001010
001011
001100
001101
001110
001111
010000
010001
010010
010011
010100
010101
010110
010111
011000
011001
011010
011011
011100
011101
011110
011111

Dividers

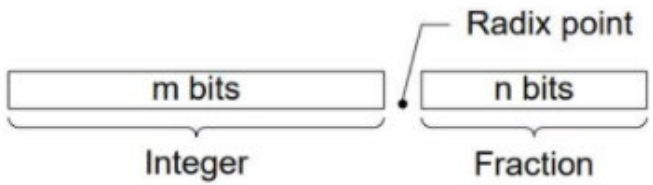
- IP Divider cores by vendors (https://www.xilinx.com/support/documentation/ip_documentation/div_gen/v5_1/pg151-div-gen.pdf)
  - **LUTMult;** A simple lookup estimate of the reciprocal of the divisor followed by a Multiplier
  - **Radix-2.** non-restoring integer division using integer operands, allowing either a fractional or an integer remainder to be generated

Fixed point vs Floating point



Fixed point

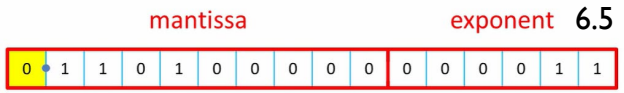
- It means seeing the radix point at the same position for all numbers
- Smaller range of numbers
- The vector ranges must be determined offline

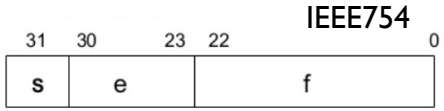


e.g, 6.5 = 0000000110.100000

Floating point

- It is used in systems has very small and very large real numbers
- Represents positive and negative numbers
- Complex logic required
- Sign, exponent, fraction;
  - 32 bits





- 754-2019 - IEEE Standard for Floating-Point Arithmetic, https://ieeexplore.ieee.org/document/8766229
- IP Core: Floating-Point Operator(IEEE 754) https://www.xilinx.com/support/documentation/ip_documentation/floating_point/v7_1/pg060-floating-point.pdf

Build a calculator

- Think within your group about how to build a calculator using the developed components so far.